Légende des types de test	
Nominal	Cas passant — comportement attendu nominal
Alternatif	Cas limite ou variation valide du nominal
Erreur	Cas rejetant — données invalides ou état incohérent
Sécurité	Contrôle d'accès et vérification des droits

PLANS DE TEST — Application Kanban (côté métier)

1 — Authentification (Deno)

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
AUTH-01	Nominal	Connexion	**Connexion avec des identifiants valides**	Compte user1 existant en base	Saisie de l'e-mail et du mot de passe de user1, soumission du formulaire	POST /api/v1/auth/login { email, password }	Token JWT retourné, objet user renvoyé (id, email, prénom, nom, isAdmin)	200
AUTH-02	Erreur	Connexion	**Mot de passe incorrect**	Compte user1 existant	Saisie de l'e-mail correct et d'un mauvais mot de passe	POST /api/v1/auth/login { email, badPassword }	Message d'erreur « Identifiants incorrects »	401
AUTH-03	Erreur	Connexion	**E-mail inconnu**	Aucun compte pour cet e-mail	Saisie d'un e-mail inexistant	POST /api/v1/auth/login { unknownEmail, pwd }	Message d'erreur « Identifiants incorrects »	401
AUTH-04	Erreur	Connexion	**Corps de requête invalide (champ manquant)**	—	Envoi du formulaire sans le champ password	POST /api/v1/auth/login { email }	Message d'erreur « Corps de requête invalide »	400
AUTH-05	Nominal	Validation token	**Vérification d'un token JWT valide**	user1 possède un token non expiré	Appel de validate avec le token en header Authorization	GET /api/v1/auth/validate Authorization: Bearer <token>	Objet user renvoyé (id, email, isAdmin)	200
AUTH-06	Erreur	Validation token	**Token JWT expiré ou falsifié**	Token périmé ou signature invalide	Appel de validate avec un token invalide	GET /api/v1/auth/validate Authorization: Bearer <badToken>	Erreur d'authentification, accès refusé	401

PLANS DE TEST — Application Kanban (côté métier)

2 — Utilisateurs

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
USR-01	Nominal	Inscription	**Création d'un nouvel utilisateur**	Adresse e-mail non utilisée	Saisie du formulaire d'inscription complet (nom, prénom, mail, mdp)	POST /api/v1/users { nom, prenom, mail, mdp }	Utilisateur créé, DTO renvoyé avec son id	201
USR-02	Erreur	Inscription	**E-mail déjà utilisé**	user1 existe déjà avec cet e-mail	Tentative d'inscription avec le même e-mail	POST /api/v1/users { mail existant }	Erreur contrainte UNIQUE, création refusée	409/500
USR-03	Nominal	Lecture profil	**Un utilisateur consulte son propre profil**	user1 connecté	Navigation vers la page profil	GET /api/v1/users/{id} X-User-Id: {id}	DTO utilisateur renvoyé (sans mdp)	200
USR-04	Sécurité	Lecture profil	**Un user tente de lire le profil d'un autre user (non admin)**	user1 connecté, tente de lire profil user2	Appel GET avec X-User-Id de user1 mais id de user2	GET /api/v1/users/{id_user2} X-User-Id: {id_user1}	Accès refusé	403
USR-05	Nominal	Mise à jour profil	**Modification de son propre profil (nom, prénom)**	user1 connecté	Formulaire de modification du profil	PUT /api/v1/users/{id} X-User-Id: {id} { nom, prenom, mail }	Profil mis à jour, DTO renvoyé	200
USR-06	Nominal	Changement mdp	**Changement de mot de passe avec l'ancien correct**	user1 connecté	Saisie ancien mdp + nouveau mdp	PATCH /api/v1/users/{id}/password X-User-Id: {id} { oldPwd, newPwd }	Mot de passe mis à jour, HTTP 204	204
USR-07	Erreur	Changement mdp	**Ancien mot de passe incorrect**	user1 connecté	Saisie d'un mauvais ancien mdp	PATCH /api/v1/users/{id}/password { wrongOldPwd, newPwd }	Erreur, changement refusé	400/403
USR-08	Nominal	Upload avatar	**Upload d'une image valide comme avatar**	user1 connecté	Sélection d'une image JPEG/PNG et upload	POST /api/v1/users/{id}/avatar multipart/form-data	Profil mis à jour avec URL avatar, DTO renvoyé	200
USR-09	Nominal	Admin : liste users	**L'administrateur liste tous les utilisateurs**	admin connecté	Navigation vers la page admin	GET /api/v1/users X-User-Id: {id_admin}	Liste complète des utilisateurs	200
USR-10	Sécurité	Admin : liste users	**Un non-admin tente de lister tous les utilisateurs**	user1 (non admin) connecté	Appel GET /users avec X-User-Id de user1	GET /api/v1/users X-User-Id: {id_user1}	Accès refusé	403

PLANS DE TEST — Application Kanban (côté métier)								
2 — Utilisateurs								
ID	**Type de test**	**Fonctionnalité**	**Description du cas**	**Préconditions**	**Actions utilisateur**	**Appel API**	**Résultat attendu**	**HTTP**
USR-11	Nominal	Admin : suppression	**L'admin supprime un utilisateur**	admin connecté, user2 existe	Clic supprimer sur user2 dans la page admin	DELETE /api/v1/users/{id_user2} X-User-Id: {id_admin}	Utilisateur supprimé, HTTP 204	204

PLANS DE TEST — Application Kanban (côté métier)

3 — Tableaux (Boards)

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
BRD-01	Nominal	Création tableau	**Création d'un tableau par user1**	user1 connecté	Saisie du nom du tableau et soumission	POST /api/v1/boards { nom, creatorId }	Tableau créé, créateur automatiquement ajouté en tant qu'admin (trigger SQL)	201
BRD-02	Nominal	Lecture tableau	**Accès au tableau par un membre actif**	user1 membre actif du tableau T1	Navigation vers le tableau T1	GET /api/v1/boards/{id} X-User-Id: {id_user1}	DTO du tableau renvoyé (id, nom, colonnes…)	200
BRD-03	Sécurité	Lecture tableau	**Accès refusé à un non-membre**	user2 n'est pas membre de T1	Tentative d'accès à T1 par user2	GET /api/v1/boards/{id} X-User-Id: {id_user2}	Accès refusé	403
BRD-04	Sécurité	Lecture tableau	**Invitation en attente = pas d'accès**	user2 invité sur T1 mais invitation non encore acceptée	Tentative d'accès à T1 par user2 (notified=false)	GET /api/v1/boards/{id} X-User-Id: {id_user2}	Accès refusé tant que l'invitation n'est pas acceptée	403
BRD-05	Nominal	Liste tableaux user	**Lister les tableaux accessibles à user1**	user1 membre de plusieurs tableaux	Chargement de la page Boards	GET /api/v1/boards/user/{id} X-User-Id: {id_user1}	Liste des tableaux avec rôle de user1 dans chacun	200
BRD-06	Nominal	Mise à jour tableau	**Renommer un tableau (admin du tableau)**	user1 admin de T1	Modification du nom du tableau	PUT /api/v1/boards/{id} { nom } X-User-Id: {id_user1}	Tableau renommé, DTO mis à jour	200
BRD-07	Sécurité	Mise à jour tableau	**Tentative de renommage par un éditeur (non admin)**	user2 éditeur de T1	Tentative de modification du nom	PUT /api/v1/boards/{id} X-User-Id: {id_user2}	Accès refusé (droits insuffisants)	403
BRD-08	Nominal	Suppression tableau	**Suppression d'un tableau par son admin**	user1 admin de T1	Clic supprimer sur T1	DELETE /api/v1/boards/{id} X-User-Id: {id_user1}	Tableau et toutes ses données supprimés (CASCADE), HTTP 204	204
BRD-09	Nominal	Export PDF	**Export PDF d'un tableau par un membre**	user1 membre actif de T1	Clic « Exporter en PDF »	GET /api/v1/boards/{id}/export X-User-Id: {id_user1}	Fichier PDF téléchargé (Content-Type: application/pdf)	200

PLANS DE TEST — Application Kanban (côté métier)

4 — Membres & Invitations

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
MBR-01	Nominal	Inviter membre	**L'admin T1 invite user2 par e-mail**	user1 admin de T1, user2 existe	Saisie de l'e-mail de user2 dans le panel membres	POST /api/v1/boards/{id}/members { mail: user2 } X-User-Id: {id_user1}	Participation créée (notified=false), e-mail envoyé à user2	201
MBR-02	Nominal	Accepter invitation	**user2 accepte son invitation**	Invitation en attente pour user2 sur T1	Clic « Accepter » dans la page Boards	PATCH /api/v1/boards/{id}/members/me/accept X-User-Id: {id_user2}	Participation mise à jour (notified=true), user2 accède au tableau	200
MBR-03	Nominal	Refuser invitation	**user2 refuse son invitation**	Invitation en attente pour user2 sur T1	Clic « Refuser »	DELETE /api/v1/boards/{id}/members/me/decline X-User-Id: {id_user2}	Participation supprimée, HTTP 204	204
MBR-04	Nominal	Changer rôle membre	**L'admin T1 passe user2 de lecteur à éditeur**	user1 admin, user2 membre actif de T1	Changement du rôle dans le panel membres	PUT /api/v1/boards/{id}/members/{id_user2}? role=editeur X-User-Id: {id_user1}	Rôle mis à jour, DTO renvoyé	200
MBR-05	Sécurité	Changer rôle membre	**Un éditeur tente de modifier un rôle**	user2 éditeur de T1	Tentative de modification du rôle de user3	PUT /api/v1/boards/{id}/members/{id_user3}? role=admin X-User-Id: {id_user2}	Accès refusé	403
MBR-06	Nominal	Retirer membre	**L'admin T1 retire user2 du tableau**	user1 admin, user2 membre de T1	Clic retirer dans le panel membres	DELETE /api/v1/boards/{id}/members/{id_user2} X-User-Id: {id_user1}	Participation supprimée, user2 n'accède plus à T1, HTTP 204	204
MBR-07	Nominal	Lister invitations en attente (admin)	**L'admin consulte les invitations non encore acceptées**	user1 admin, invitation en attente pour user3	Consultation de la liste pending dans le panel admin	GET /api/v1/boards/{id}/members/pending X-User-Id: {id_user1}	Liste des membres avec notified=false	200
MBR-08	Nominal	Valider manuellement invitation (admin)	**L'admin valide manuellement l'invitation de user3**	Invitation pending pour user3	Clic « Valider » dans le panel admin	PATCH /api/v1/boards/{id}/members/{id_user3} /validate X-User-Id: {id_user1}	notified=true, user3 accède au tableau	200

PLANS DE TEST — Application Kanban (côté métier)

5 — Colonnes (BoardColumns)

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
COL-01	Nominal	Création colonne	**Ajout d'une colonne à un tableau**	user1 membre de T1	Clic « + colonne » et saisie du nom	POST /api/v1/boards/{id}/columns { nom } X-User-Id: {id_user1}	Colonne créée, col_numero auto-incrémenté (trigger SQL)	201
COL-02	Nominal	Liste colonnes	**Consultation des colonnes du tableau**	user1 membre actif de T1	Ouverture du tableau T1	GET /api/v1/boards/{id}/columns X-User-Id: {id_user1}	Liste triée par col_numero croissant	200
COL-03	Nominal	Renommer colonne	**Renommage d'une colonne existante**	user1 membre de T1, colonne C1 existe	Double-clic sur le nom de C1, saisie du nouveau nom	PUT /api/v1/boards/{id}/columns/{col_id} { nom } X-User-Id: {id_user1}	Colonne renommée, DTO mis à jour	200
COL-04	Nominal	Déplacer colonne	**Déplacement d'une colonne vers une nouvelle position**	T1 possède 3 colonnes numérotées 1, 2, 3	Drag & drop de la colonne 1 vers la position 3	PATCH /api/v1/boards/{id}/columns/{col_id} /move?newNum=3 X-User-Id: {id_user1}	Renumérotage cohérent : 2→1, 3→2, cible→3 (procédure SQL deplacer_colonne)	204
COL-05	Alternatif	Déplacer colonne	**Position cible > max existant → borne au max**	T1 possède 3 colonnes	Tentative de déplacement vers position 99	PATCH /api/v1/boards/{id}/columns/{col_id} /move?newNum=99 X-User-Id: {id_user1}	Colonne placée en dernière position (3), pas d'erreur	204
COL-06	Erreur	Modification directe col_numero	**Tentative de modification directe de col_numero (contournement)**	—	UPDATE direct SQL ou appel API non prévu	UPDATE colonne_col SET col_numero=5 WHERE col_id=1	Trigger before_update_colonne lève une erreur SQL 45000	500
COL-07	Nominal	Suppression colonne	**Suppression d'une colonne et renumérotage**	T1 possède colonnes 1, 2, 3 — suppression de la colonne 2	Clic « supprimer » sur la colonne 2	DELETE /api/v1/boards/{id}/columns/{col_id} X-User-Id: {id_user1}	Colonne et ses tâches supprimées (CASCADE), colonne 3 renumérotée en 2	204
COL-08	Sécurité	Accès colonnes	**Non-membre tente de lister les colonnes**	user2 non membre de T1	Appel GET colonnes de T1	GET /api/v1/boards/{id}/columns X-User-Id: {id_user2}	Accès refusé	403

PLANS DE TEST — Application Kanban (côté métier)

6 — Tâches (Tasks)

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
TSK-01	Nominal	Création tâche	**Création d'une tâche dans une colonne**	user1 membre de T1, colonne C1 existe	Clic « + tâche », saisie titre, priorité, date limite	POST /api/v1/tasks { titre, colId, priorite, dateLimite } X-User-Id: {id_user1}	Tâche créée en fin de colonne (trigger before_insert_tache), log 'creation' généré	201
TSK-02	Nominal	Lecture tâche	**Consultation d'une tâche par son ID**	user1 membre, tâche TK1 dans T1	Clic sur la tâche pour ouvrir le détail	GET /api/v1/tasks/{id} X-User-Id: {id_user1}	DTO de la tâche renvoyé	200
TSK-03	Nominal	Liste tâches colonne	**Lister les tâches d'une colonne triées par position**	user1 membre, C1 contient 3 tâches	Affichage de la colonne C1	GET /api/v1/tasks/column/{col_id} X-User-Id: {id_user1}	Liste des tâches triée par tsk_position croissant	200
TSK-04	Nominal	Mise à jour tâche	**Modification du titre et de la priorité d'une tâche**	user1 membre, tâche TK1 existe	Ouverture du modal, modification titre + priorité	PUT /api/v1/tasks/{id} { titre, priorite, details } X-User-Id: {id_user1}	Tâche mise à jour, log 'modification' généré, DTO renvoyé	200
TSK-05	Nominal	Déplacement intra-colonne	**Réordonner une tâche dans la même colonne**	C1 contient tâches positions 0, 1, 2 — déplacer position 0 en position 2	Drag & drop de TK0 vers la position 2 dans C1	PATCH /api/v1/tasks/{id}/move?newId={col_id} &position=2 X-User-Id: {id_user1}	Positions réarrangées : 1→0, 2→1, TK0→2 ; log 'deplacement' généré	200
TSK-06	Nominal	Déplacement inter-colonnes	**Déplacer une tâche vers une autre colonne**	TK1 dans C1 (position 0), C2 contient 2 tâches	Drag & drop de TK1 de C1 vers C2	PATCH /api/v1/tasks/{id}/move?newId= {col_id_c2} X-User-Id: {id_user1}	TK1 déplacé en fin de C2 ; trou fermé dans C1 ; log 'deplacement' généré	200
TSK-07	Erreur	Déplacement inter-tableaux	**Tentative de déplacement vers une colonne d'un autre tableau**	TK1 dans tableau T1, tentative vers colonne de T2	Appel move avec col_id appartenant à T2	PATCH /api/v1/tasks/{id}/move?newId= {col_T2_id} X-User-Id: {id_user1}	Erreur : colonne cible dans un tableau différent	400
TSK-08	Nominal	Assignation	**Assigner une tâche à user2**	user1 membre T1, user2 membre T1, TK1 non assignée	Sélection de user2 dans le sélecteur d'assignation	PATCH /api/v1/tasks/{id}/assign?userId= {id_user2} X-User-Id: {id_user1}	Tâche assignée à user2, log 'modification' généré	200
TSK-09	Nominal	Désassignation	**Retirer l'assignation d'une tâche**	TK1 assignée à user2	Clic pour retirer l'assignation	PATCH /api/v1/tasks/{id}/assign (sans userId) X-User-Id: {id_user1}	usr_assignee mis à NULL, log 'modification' généré	200
TSK-10	Nominal	Mes tâches	**user2 consulte ses tâches assignées**	user2 a plusieurs tâches assignées	Navigation vers la page « Mes tâches »	GET /api/v1/tasks/assigned/{id_user2}	Liste de toutes les tâches assignées à user2	200

PLANS DE TEST — Application Kanban (côté métier)

6 — Tâches (Tasks)

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
TSK-11	Nominal	Recherche tâches	**Recherche par mot-clé dans un tableau**	user1 membre T1, tâches contenant « bug »	Saisie de « bug » dans la barre de recherche	GET /api/v1/tasks/search?boardId={id} &keyword=bug X-User-Id: {id_user1}	Liste des tâches dont le titre ou les détails contiennent « bug »	200
TSK-12	Nominal	Recherche par priorité	**Filtrer les tâches de priorité haute**	user1 membre T1	Sélection du filtre priorité=haute	GET /api/v1/tasks/search?boardId={id} &priority=haute X-User-Id: {id_user1}	Liste des tâches haute priorité triée par colonne/position	200
TSK-13	Nominal	Validation tâche	**Marquer une tâche comme validée**	user1 membre, TK1 non validée	Clic sur la case à cocher de la tâche	PUT /api/v1/tasks/{id} { validee: true } X-User-Id: {id_user1}	tsk_validee=true, log 'modification' généré	200
TSK-14	Nominal	Suppression tâche	**Suppression d'une tâche**	user1 membre, TK1 existe	Clic « supprimer » sur TK1	DELETE /api/v1/tasks/{id} X-User-Id: {id_user1}	Tâche et ses commentaires supprimés (MongoDB + MariaDB), HTTP 204	204
TSK-15	Sécurité	Accès tâche	**Non-membre tente d'accéder à une tâche**	user2 non membre de T1	Appel GET tâche appartenant à T1	GET /api/v1/tasks/{id} X-User-Id: {id_user2}	Accès refusé	403
TSK-16	Alternatif	Tsk position ≥ max	**Position cible > max → borne automatique**	C1 contient 3 tâches (pos 0,1,2)	Déplacement vers position 99	PATCH /api/v1/tasks/{id}/move?newId={col_id} &position=99 X-User-Id: {id_user1}	Tâche placée en dernière position (2) sans erreur	200

PLANS DE TEST — Application Kanban (côté métier)

7 — Commentaires (MongoDB)

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
CMT-01	Nominal	Ajout commentaire	**Ajouter un commentaire à une tâche**	user1 membre T1, TK1 existe	Saisie du texte dans la zone commentaire et envoi	POST /api/v1/comments { tskId, usrId, contenu }	Commentaire créé dans MongoDB, DTO renvoyé avec ObjectId	201
CMT-02	Nominal	Liste commentaires	**Lister les commentaires d'une tâche triés chronologiquement**	TK1 possède 3 commentaires	Ouverture du détail de TK1	GET /api/v1/comments/task/{tsk_id}	Liste triée par date croissante	200
CMT-03	Nominal	Modification commentaire	**Modifier le texte d'un commentaire**	Commentaire C1 existe (ObjectId connu)	Clic « modifier », saisie du nouveau texte	PUT /api/v1/comments/{objectId} { contenu }	Commentaire mis à jour, DTO renvoyé	200
CMT-04	Nominal	Suppression commentaire	**Supprimer un commentaire**	Commentaire C1 existe	Clic « supprimer » sur le commentaire	DELETE /api/v1/comments/{objectId}	Commentaire supprimé de MongoDB, HTTP 204	204
CMT-05	Alternatif	Suppression tâche → commentaires	**La suppression de la tâche supprime ses commentaires MongoDB**	TK1 a 3 commentaires dans MongoDB	Suppression de TK1 via l'API	DELETE /api/v1/tasks/{id} X-User-Id: {id_user1}	TK1 supprimée ET ses 3 commentaires supprimés de MongoDB	204

PLANS DE TEST — Application Kanban (côté métier)

8 — Journal d'activité (Logs)

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
LOG-01	Nominal	Logs tableau	**Consulter le journal d'un tableau**	user1 membre T1, plusieurs actions effectuées	Navigation vers la page Logs du tableau	GET /api/v1/boards/{id}/logs X-User-Id: {id_user1}	Liste des logs triée par date décroissante	200
LOG-02	Nominal	Logs tâche	**Consulter le journal d'une tâche**	TK1 a été créée, déplacée et modifiée	Ouverture de l'historique de TK1	GET /api/v1/tasks/{id}/logs X-User-Id: {id_user1}	3 entrées : creation, deplacement, modification	200
LOG-03	Nominal	Logs utilisateur	**Un user consulte son propre journal**	user1 a effectué plusieurs actions	Navigation vers la page Logs utilisateur	GET /api/v1/users/{id}/logs X-User-Id: {id_user1}	Journal des actions de user1	200
LOG-04	Sécurité	Logs utilisateur	**user1 tente de consulter le journal de user2**	user1 non admin	Appel GET logs de user2 avec X-User-Id de user1	GET /api/v1/users/{id_user2}/logs X-User-Id: {id_user1}	Accès refusé	403
LOG-05	Nominal	Filtre type log	**Filtrer les logs par type 'creation'**	T1 a des logs de différents types	Sélection du filtre type=creation	GET /api/v1/boards/{id}/logs?type=creation X-User-Id: {id_user1}	Uniquement les logs de type 'creation'	200
LOG-06	Nominal	Filtre plage de dates	**Filtrer les logs entre deux dates**	T1 a des logs sur plusieurs jours	Saisie d'une plage de dates dans le filtre	GET /api/v1/boards/{id}/logs?from=...&to=... X-User-Id: {id_user1}	Logs dans la plage de dates uniquement	200
LOG-07	Nominal	Log création auto (trigger)	**Vérification que la création de tâche génère un log**	Aucun log pour TK1 avant création	Création de la tâche TK1	POST /api/v1/tasks → vérifié par GET /api/v1/tasks/{id}/logs	1 entrée log_type='creation' dans journal_log	200
LOG-08	Nominal	Log déplacement auto (trigger)	**Vérification que le déplacement génère un seul log**	TK1 existe dans C1	Déplacement de TK1 vers C2 via deplacer_tache	PATCH /api/v1/tasks/{id}/move → vérifié par GET /api/v1/tasks/{id}/logs	1 entrée log_type='deplacement', pas de logs parasites sur les tâches voisines	200

PLANS DE TEST — Application Kanban (côté métier)

9 — Contraintes BDD / Triggers SQL

ID	Type de test	Fonctionnalité	Description du cas	Préconditions	Actions utilisateur	Appel API	Résultat attendu	HTTP
SQL-01	Nominal	Trigger after_insert_tableau	**Création auto de la participation admin du créateur**	—	Création du tableau T1 par user1	POST /api/v1/boards → vérification en SELECT participation_ptp	Ligne dans participation_ptp avec role='admin' et notified=true pour user1	201
SQL-02	Nominal	Trigger before_insert_colonne	**Auto-incrémentation de col_numero**	T1 possède déjà 2 colonnes	Création d'une 3e colonne sans spécifier col_numero	POST /api/v1/boards/{id}/columns { nom }	col_numero=3 attribué automatiquement	201
SQL-03	Nominal	Trigger before_insert_tache	**Auto-incrémentation de tsk_position**	C1 contient déjà 2 tâches (positions 0 et 1)	Création d'une 3e tâche	POST /api/v1/tasks { colId: C1, titre }	tsk_position=2 attribué automatiquement	201
SQL-04	Nominal	Contrainte UNIQUE tsk_position	**Pas de doublon de position dans une colonne**	C1 a une tâche en position 0	Tentative d'insertion d'une 2e tâche en position 0	INSERT direct SQL ou via API avec @_disable_tache_triggers=1	Violation contrainte uq_tsk_position_col	409/500
SQL-05	Nominal		**Suppression du tableau supprime colonnes, tâches et logs**	T1 a 2 colonnes, 5 tâches, 3 logs	Suppression de T1	DELETE /api/v1/boards/{id} → vérification COUNT(*) dans chaque table	Toutes les colonnes, tâches et logs liés à T1 supprimés	204
SQL-06	Nominal	SET NULL sur suppression user assigné	**Suppression de l'utilisateur assigné met usr_assignee à NULL**	TK1 assignée à user2	Suppression de user2 par admin	DELETE /api/v1/users/{id_user2} → SELECT tsk_assignee FROM tache_tsk	usr_assignee=NULL sur TK1, la tâche existe toujours	204